

Guide descriptif et guide d'utilisation de l'application SGRH Intelligent

MANSOUR Aymane FOUKOU Issa BADIA Salahedine

Année universitaire 2025–2026

Table des matières

Introduction	2
1 Présentation générale de l'application	3
1.1 Objectif de l'application	3
1.2 Architecture globale	4
1.3 Technologies utilisées	4
2 Gestion de l'authentification	5
2.1 Description de l'interface de connexion	5
2.2 Fichiers frontend associés	5
2.3 Fichiers backend associés	6
3 Description détaillée des interfaces	7
3.1 Tableau de bord	7
3.1.1 Rôle de l'interface	7
3.1.2 Fichiers en arrière-plan	7
3.2 Interface de gestion des employés	8
3.2.1 Rôle de l'interface	8
3.2.2 Actions disponibles	8
3.2.3 Fichiers en arrière-plan	8
3.3 Interface de gestion des congés	9
3.3.1 Rôle de l'interface	9
3.3.2 Actions disponibles	9
3.3.3 Fichiers en arrière-plan	9
3.4 Interface de gestion des présences	10
3.4.1 Rôle de l'interface	10
3.4.2 Fichiers en arrière-plan	10
3.5 Interface de gestion de la paie	10
3.5.1 Rôle de l'interface	10
3.5.2 Fichiers en arrière-plan	11
3.6 Interface Documents et OCR	11
3.6.1 Rôle de l'interface	11
3.6.2 Actions disponibles	11
3.6.3 Fichiers en arrière-plan	12
3.7 Interface Analyse IA	12
3.7.1 Rôle de l'interface	12
3.7.2 Fichiers en arrière-plan	13
3.8 Interface de gestion des utilisateurs	13
3.8.1 Rôle de l'interface	13

3.8.2	Fichiers en arrière-plan	13
4	Navigation et sécurité des interfaces	15
4.1	Gestion des routes	15
4.2	Menu latéral	15
5	Guide de lancement de l'application	17
5.1	Prérequis	17
5.2	Préparation de la base de données	17
5.3	Lancement du microservice IA	18
5.4	Lancement du backend Spring Boot	18
5.5	Lancement du frontend React	19
5.6	Ordre recommandé de lancement	19
6	Guide d'utilisation de l'application	20
6.1	Connexion	20
6.2	Utilisation par un administrateur	20
6.3	Utilisation par un responsable RH	21
6.4	Utilisation par un employé	21
6.5	Bonnes pratiques d'utilisation	21
7	Conclusion	22

Introduction

Ce document présente en détail l'application **SGRH Intelligent**, un système de gestion des ressources humaines conçu pour digitaliser les opérations administratives liées aux employés, aux congés, aux présences, à la paie et aux documents. Dans un contexte professionnel où la rapidité de traitement, la fiabilité des informations et la centralisation des données deviennent des exigences essentielles, cette application propose une solution moderne permettant de remplacer une partie importante des traitements manuels par des processus informatisés, plus simples à contrôler et plus faciles à exploiter.

Le projet ne se limite pas à une simple application de gestion classique. Il intègre également des fonctionnalités avancées basées sur l'intelligence artificielle pour la détection d'anomalies et l'analyse comportementale, ainsi qu'un module OCR destiné au traitement automatique des documents administratifs. Cette combinaison entre gestion RH, analyse intelligente et extraction automatique de texte apporte une valeur ajoutée importante, car elle permet non seulement d'enregistrer les données, mais aussi de les exploiter afin d'aider les responsables RH dans la prise de décision.

Dans le cadre d'un Projet de Fin d'Études, la réalisation de cette application permet de mettre en pratique plusieurs compétences techniques : conception d'une architecture web complète, sécurisation des accès, développement d'interfaces utilisateur ergonomiques, mise en place d'une API REST, gestion d'une base de données relationnelle et intégration de services externes tels que l'OCR et le microservice IA. Le système obtenu représente ainsi une application complète, modulaire et évolutive.

L'objectif de ce guide est triple :

- décrire le fonctionnement général de l'application ;
- expliquer le rôle de chaque interface utilisateur ;
- présenter les fichiers frontend, backend et microservice qui fonctionnent en arrière-plan pour chaque module ;
- fournir un guide pratique pour lancer et utiliser l'application.

Chapitre 1

Présentation générale de l'application

1.1 Objectif de l'application

L'application SGRH Intelligent a pour objectif de centraliser la gestion des ressources humaines au sein d'une organisation. Dans de nombreuses structures, les informations relatives aux employés, aux congés, aux présences ou à la paie sont souvent dispersées entre plusieurs fichiers, documents papier ou outils indépendants. Cette dispersion rend le suivi difficile, augmente le risque d'erreurs et complique la recherche rapide d'informations. Le système développé vise donc à regrouper ces données dans une seule plateforme accessible, organisée et sécurisée.

L'application permet aux administrateurs, aux responsables RH et aux employés d'accéder à des fonctionnalités adaptées à leurs rôles respectifs. Cette séparation des rôles est importante, car tous les utilisateurs ne doivent pas avoir le même niveau d'accès aux données sensibles. Par exemple, un administrateur peut gérer les comptes utilisateurs, tandis qu'un employé peut uniquement consulter certaines informations ou déposer une demande de congé. Cette logique garantit une meilleure confidentialité des données et une utilisation plus contrôlée de l'application.

Un autre objectif important du projet est l'amélioration de l'efficacité administrative. Les opérations répétitives, comme l'enregistrement d'un employé, le suivi d'une demande de congé ou la consultation des présences, sont simplifiées grâce à des interfaces claires. L'utilisateur n'a pas besoin de manipuler directement la base de données ni de comprendre la logique technique interne : il interagit avec des formulaires, des tableaux, des boutons d'action et des messages de confirmation.

Les principales fonctionnalités sont :

- authentification sécurisée des utilisateurs ;
- consultation du tableau de bord RH ;
- gestion des employés ;
- gestion des demandes de congés ;
- suivi des présences et des retards ;
- gestion de la paie ;
- gestion des documents administratifs avec OCR ;
- analyse IA pour la détection d'anomalies ;
- gestion des comptes utilisateurs.

1.2 Architecture globale

Le système est organisé en plusieurs couches afin de séparer clairement les responsabilités de chaque partie de l'application. Cette séparation facilite la maintenance, car une modification de l'interface n'implique pas nécessairement une modification de la logique métier. Elle facilite également l'évolution du projet, puisqu'il devient possible d'ajouter de nouveaux modules sans remettre en cause toute l'architecture existante.

Le frontend représente la partie visible par l'utilisateur. Il est chargé d'afficher les pages, les tableaux, les formulaires, les modales et les graphiques. Le backend constitue la partie métier de l'application. Il reçoit les requêtes envoyées par le frontend, vérifie les autorisations, applique les règles de gestion et communique avec la base de données. La base MySQL conserve les données de manière structurée et persistante. Enfin, le microservice IA fonctionne comme un service indépendant spécialisé dans les traitements intelligents.

Cette architecture permet d'obtenir une application plus robuste. Si le microservice IA n'est pas disponible, les modules classiques de gestion RH peuvent continuer à fonctionner. De même, si l'interface frontend évolue, le backend peut conserver les mêmes endpoints API. Cette indépendance entre les couches constitue un choix technique adapté aux applications professionnelles modernes.

Le système est organisé en plusieurs composants principaux :

- **Frontend React** : responsable des interfaces graphiques accessibles depuis le navigateur ;
- **Backend Spring Boot** : responsable de la logique métier, de la sécurité et de l'exposition des API REST ;
- **Base de données MySQL** : responsable du stockage des utilisateurs, employés, congés, présences, documents et anomalies ;
- **Microservice IA FastAPI** : responsable des traitements d'intelligence artificielle ;
- **Tesseract OCR** : utilisé pour l'extraction automatique de texte depuis les documents.

1.3 Technologies utilisées

Composant	Technologies
Frontend	React, Vite, Tailwind CSS, React Router, Axios, Recharts, Lucide React
Backend	Java 21, Spring Boot, Spring Security, Spring Data JPA, JWT, Maven
Base de données	MySQL 8
Microservice IA	Python 3.11, FastAPI, Uvicorn, Scikit-learn, Pandas, NumPy
OCR	Tesseract OCR

Chapitre 2

Gestion de l'authentification

2.1 Description de l'interface de connexion

L'interface de connexion constitue le premier point de contact entre l'utilisateur et l'application. Elle joue un rôle essentiel, car elle permet de vérifier l'identité de chaque utilisateur avant de lui donner accès aux fonctionnalités internes du système. L'utilisateur doit saisir son adresse email ainsi que son mot de passe. Ces informations sont ensuite envoyées au backend, qui se charge de vérifier leur validité à l'aide du mécanisme d'authentification configuré avec Spring Security.

Cette page a été conçue pour être simple, claire et intuitive. Elle contient un champ pour l'adresse email, un champ pour le mot de passe et un bouton de connexion. Une option permet également d'afficher ou de masquer le mot de passe, ce qui améliore l'expérience utilisateur tout en conservant un niveau de confidentialité acceptable. En cas d'erreur, par exemple si les identifiants sont incorrects, un message est affiché afin d'informer l'utilisateur de l'échec de la connexion.

L'interface propose aussi un mode de démonstration permettant de se connecter rapidement avec différents rôles : administrateur, responsable RH ou employé. Ce mode est particulièrement utile pendant la phase de présentation ou de test du projet, car il permet de montrer rapidement les différences d'accès entre les profils sans devoir saisir manuellement les identifiants.

Après une authentification réussie, l'utilisateur est redirigé vers le tableau de bord. Le backend retourne un token JWT qui est conservé côté frontend. Ce token accompagne ensuite les requêtes envoyées vers les API protégées. Le rôle de l'utilisateur détermine les pages accessibles dans le menu latéral et empêche un utilisateur non autorisé d'accéder à certaines fonctionnalités sensibles, comme la gestion des utilisateurs ou la gestion de la paie.

2.2 Fichiers frontend associés

- `frontend/src/pages/LoginPage.jsx` : contient le formulaire de connexion, la gestion du mot de passe visible ou masqué, l'appel au service d'authentification et le mode démo.
- `frontend/src/context/AuthContext.jsx` : conserve l'utilisateur connecté et le token JWT dans le contexte global de l'application.

- `frontend/src/services/api.js` : contient `authService.login`, `authService.register` et l'intercepteur `Axios` qui ajoute le token `JWT` aux requêtes.
- `frontend/src/App.jsx` : protège les routes grâce au composant `ProtectedRoute`.

2.3 Fichiers backend associés

- `backend/src/main/java/ma/fsb/sgrh/controller/AuthController.java` : expose les endpoints `/api/auth/login` et `/api/auth/register`.
- `backend/src/main/java/ma/fsb/sgrh/security/JwtService.java` : génère et valide les tokens `JWT`.
- `backend/src/main/java/ma/fsb/sgrh/security/SecurityConfig.java` : configure `Spring Security`, les permissions, le filtre `JWT` et le chiffrement des mots de passe.
- `backend/src/main/java/ma/fsb/sgrh/security/JwtAuthFilter.java` : intercepte les requêtes `HTTP` pour vérifier le token.
- `backend/src/main/java/ma/fsb/sgrh/entity/Utilisateur.java` : représente l'utilisateur dans la base de données.

Chapitre 3

Description détaillée des interfaces

3.1 Tableau de bord

3.1.1 Rôle de l'interface

Le tableau de bord est la page principale affichée après connexion. Il représente une synthèse générale de l'activité RH et permet à l'utilisateur d'obtenir rapidement une vision globale de l'état du système. Au lieu de consulter séparément chaque module, l'utilisateur retrouve sur cette page les indicateurs les plus importants : nombre total d'employés, demandes de congés en attente, présences du jour, anomalies détectées et activités récentes.

Cette interface sert donc de point d'entrée fonctionnel pour les responsables RH et les administrateurs. Elle leur permet d'identifier rapidement les éléments qui nécessitent une intervention, comme une demande de congé non traitée ou une anomalie détectée. Les indicateurs sont présentés sous forme de cartes statistiques et de graphiques afin de rendre l'information plus lisible. Ce choix visuel permet de réduire le temps nécessaire à l'analyse des données et améliore la prise de décision.

Le tableau de bord joue également un rôle de valorisation du projet. Il affiche les informations relatives aux réalisateurs de l'application et à l'encadrement pédagogique. Cette partie permet d'intégrer l'identité du projet directement dans l'application, ce qui est utile dans le contexte d'une soutenance ou d'une démonstration académique.

3.1.2 Fichiers en arrière-plan

- `frontend/src/pages/DashboardPage.jsx` : construit l'interface du tableau de bord, les cartes statistiques, les graphiques et les activités récentes.
- `frontend/src/services/api.js` : contient `dashboardService.getStats()` pour récupérer les statistiques depuis le backend.
- `backend/src/main/java/ma/fsb/sgrh/controller/DashboardController.java` : expose `/api/dashboard/stats` et calcule les indicateurs principaux.
- `backend/src/main/java/ma/fsb/sgrh/repository/EmployeeRepository.java` : permet de compter les employés.
- `backend/src/main/java/ma/fsb/sgrh/repository/CongeRepository.java` : permet de récupérer les congés en attente.

- `backend/src/main/java/ma/fsb/sgrh/repository/PresenceRepository.java` : permet de compter les présences du jour.
- `backend/src/main/java/ma/fsb/sgrh/repository/AnomalieRepository.java` : permet de compter les anomalies détectées.

3.2 Interface de gestion des employés

3.2.1 Rôle de l'interface

L'interface de gestion des employés est l'un des modules centraux de l'application, car l'employé constitue l'entité principale autour de laquelle s'organisent les autres fonctionnalités RH. Chaque demande de congé, chaque présence, chaque document ou fiche de paie peut être associée à un employé. Il est donc nécessaire de disposer d'une interface fiable permettant de consulter, ajouter, modifier ou supprimer ces informations.

Cette interface permet aux responsables RH et aux administrateurs de consulter la liste des employés enregistrés. Les données sont présentées sous forme de tableau afin de faciliter la lecture et la comparaison entre les enregistrements. Les informations principales affichées sont le nom, le prénom, le matricule, le poste, le département, le salaire et le statut de l'employé. Le statut permet notamment de distinguer les employés actifs des employés inactifs.

Elle permet également d'ajouter un nouvel employé, de visualiser ses détails, de modifier ses informations et de supprimer un employé après confirmation. L'ajout s'effectue à travers un formulaire qui regroupe les informations nécessaires à l'enregistrement. La modification utilise une fenêtre préremplie afin d'éviter à l'utilisateur de ressaisir toutes les informations. La suppression est protégée par une demande de confirmation pour éviter les erreurs de manipulation.

Une barre de recherche permet de filtrer les employés par nom, matricule ou département. Cette fonctionnalité devient importante lorsque le nombre d'employés augmente, car elle évite à l'utilisateur de parcourir manuellement tout le tableau. L'ensemble de ces actions contribue à rendre la gestion du personnel plus rapide, plus organisée et plus sécurisée.

3.2.2 Actions disponibles

- **Ajouter** : ouvre un formulaire de création d'un nouvel employé ;
- **Visualiser** : affiche les informations détaillées de l'employé ;
- **Modifier** : ouvre un formulaire prérempli permettant la mise à jour ;
- **Supprimer** : demande une confirmation avant suppression ;
- **Rechercher** : filtre la liste affichée.

3.2.3 Fichiers en arrière-plan

- `frontend/src/pages/EmployeesPage.jsx` : contient l'interface, le tableau, la recherche et les modales d'ajout, de visualisation et de modification.
- `frontend/src/services/api.js` : contient `employeeService.getAll`, `getById`, `create`, `update` et `delete`.

- `backend/src/main/java/ma/fsb/sgrh/controller/EmployeeController.java` : expose les endpoints REST pour créer, lire, modifier et supprimer des employés.
- `backend/src/main/java/ma/fsb/sgrh/entity/Employee.java` : définit la structure d'un employé.
- `backend/src/main/java/ma/fsb/sgrh/repository/EmployeeRepository.java` : assure l'accès aux données des employés.

3.3 Interface de gestion des congés

3.3.1 Rôle de l'interface

L'interface des congés permet de gérer le cycle complet d'une demande de congé, depuis sa création jusqu'à sa validation ou son refus. Ce module répond à un besoin fréquent dans les services RH, car le traitement manuel des congés peut être long et source d'erreurs, notamment lorsqu'il faut vérifier les dates, les soldes disponibles et l'historique des demandes.

Les employés peuvent soumettre une demande en indiquant la date de début, la date de fin, le type de congé et le motif. Chaque demande possède ensuite un statut qui indique son état actuel : en attente, validée ou refusée. Cette notion de statut permet de suivre facilement l'avancement du traitement et d'éviter les confusions entre les demandes déjà traitées et celles qui nécessitent encore une décision.

Les responsables RH et les administrateurs peuvent valider ou refuser les demandes en attente. Lorsqu'une demande est validée, le solde de congé de l'employé peut être décrémenté côté backend. Cette automatisation permet de garantir la cohérence des données et de réduire les interventions manuelles. Le module contribue ainsi à une meilleure traçabilité des absences et à une gestion plus transparente des droits aux congés.

3.3.2 Actions disponibles

- **Nouvelle demande** : création d'une demande de congé ;
- **Filtrage** : affichage par statut : tous, en attente, validé ou refusé ;
- **Valider** : accepte une demande en attente ;
- **Refuser** : refuse une demande en attente.

3.3.3 Fichiers en arrière-plan

- `frontend/src/pages/CongesPage.jsx` : affiche la liste des congés, les filtres et le formulaire de création.
- `frontend/src/services/api.js` : contient `congeService.getAll`, `getByEmploye`, `create`, `valider` et `refuser`.
- `backend/src/main/java/ma/fsb/sgrh/controller/CongeController.java` : expose les endpoints de gestion des congés.
- `backend/src/main/java/ma/fsb/sgrh/entity/Conge.java` : représente une demande de congé.
- `backend/src/main/java/ma/fsb/sgrh/entity/StatutConge.java` : définit les statuts possibles.

- `backend/src/main/java/ma/fsb/sgrh/repository/CongeRepository.java` : permet de rechercher les congés par employé ou par statut.

3.4 Interface de gestion des présences

3.4.1 Rôle de l'interface

L'interface des présences permet de suivre les pointages des employés au cours d'une journée donnée. Elle répond au besoin de contrôler la ponctualité, la présence effective et les éventuels retards. Dans une organisation, le suivi des présences est un élément important, car il peut influencer la paie, l'évaluation de l'assiduité et la détection de comportements inhabituels.

La page affiche les heures d'arrivée et de départ, la durée de présence calculée, ainsi que le statut de ponctualité. Lorsqu'un employé arrive après l'heure limite définie, il peut être marqué comme étant en retard. Cette information peut ensuite être utilisée par les responsables RH pour analyser les retards répétés ou produire des rapports statistiques.

L'interface propose aussi des cartes statistiques indiquant le nombre de présents, d'absents, de retards et le taux de présence. Ces indicateurs donnent une vision immédiate de la situation journalière. Ils sont utiles pour identifier rapidement les problèmes de présence et pour suivre l'évolution du taux d'assiduité au sein de l'organisation.

3.4.2 Fichiers en arrière-plan

- `frontend/src/pages/PresencesPage.jsx` : affiche les statistiques et le tableau des pointages.
- `frontend/src/services/api.js` : contient `presenceService.getAll`, `getByEmploye` et `pointer`.
- `backend/src/main/java/ma/fsb/sgrh/controller/PresenceController.java` : expose les endpoints de consultation et de pointage.
- `backend/src/main/java/ma/fsb/sgrh/entity/Presence.java` : représente une présence.
- `backend/src/main/java/ma/fsb/sgrh/repository/PresenceRepository.java` : permet de rechercher les présences par employé ou par date.

3.5 Interface de gestion de la paie

3.5.1 Rôle de l'interface

L'interface de paie est destinée aux administrateurs et aux responsables RH. Elle permet de consulter les fiches de paie générées pour les employés, avec les informations relatives au salaire de base, aux primes, aux retenues et au salaire net. La paie étant un domaine sensible, cette interface est protégée par des restrictions d'accès afin que seuls les utilisateurs autorisés puissent la consulter.

Elle sert à centraliser les documents de paie et à faciliter le suivi mensuel des rémunérations. L'objectif est de regrouper les informations salariales dans un espace unique, afin d'éviter les fichiers dispersés et les erreurs de calcul. Dans une version évoluée, ce

module peut être connecté aux présences, aux absences et aux primes afin de produire automatiquement les fiches de paie mensuelles.

Du point de vue de l'utilisateur, l'intérêt principal est de pouvoir consulter rapidement les informations financières d'un employé et vérifier les éléments composant son salaire. Pour l'administration, ce module offre une meilleure traçabilité des rémunérations et facilite l'archivage des données liées à la paie.

3.5.2 Fichiers en arrière-plan

- `frontend/src/pages/PaiePage.jsx` : affiche les fiches de paie et les informations salariales.
- `frontend/src/services/api.js` : contient `paieService.getAll`, `getByEmploye` et `generer`.
- `backend/src/main/java/ma/fsb/sgrh/entity/FichePaie.java` : représente une fiche de paie.
- `backend/src/main/java/ma/fsb/sgrh/repository/FichePaieRepository.java` : permet l'accès aux fiches de paie stockées.

3.6 Interface Documents et OCR

3.6.1 Rôle de l'interface

L'interface Documents et OCR permet de gérer les documents administratifs liés aux employés. Dans un service RH, plusieurs types de documents doivent être conservés : certificats médicaux, contrats de travail, attestations, diplômes, justificatifs et autres fichiers administratifs. La gestion manuelle de ces documents peut rapidement devenir complexe, surtout lorsqu'ils sont stockés sous forme papier ou répartis dans plusieurs dossiers numériques.

Le module proposé permet de déposer ces documents directement dans l'application. L'utilisateur peut sélectionner un fichier au format PDF, PNG ou JPG, puis préciser son type. Une fois le document envoyé, il peut être sauvegardé côté serveur et associé à des métadonnées comme le nom du fichier, la date d'envoi, le type de document et son statut de traitement.

L'intégration de l'OCR constitue un avantage important. Grâce à Tesseract OCR, le système peut extraire automatiquement le texte contenu dans un document scanné ou une image. Cette fonctionnalité permet de transformer un document non structuré en texte exploitable. Par exemple, un certificat médical peut être analysé afin d'extraire son contenu et faciliter son traitement par le service RH.

L'interface permet aussi de consulter les détails d'un document, de visualiser le texte extrait et de supprimer un document. Le statut du document indique si le traitement OCR est terminé, en cours ou s'il a rencontré une erreur. Cette organisation permet de suivre clairement l'état des fichiers et d'améliorer l'archivage administratif.

3.6.2 Actions disponibles

- **Uploader un document** : sélection d'un fichier PDF, PNG ou JPG ;
- **Visualiser** : affichage des détails du document ;

- **Supprimer** : suppression après confirmation ;
- **Suivre le statut** : traité, en cours ou erreur.

3.6.3 Fichiers en arrière-plan

- `frontend/src/pages/DocumentsPage.jsx` : contient l'interface de dépôt, les cartes de documents et la modale de détail.
- `frontend/src/services/api.js` : contient `documentService.getAll`, `upload` et `getById`.
- `backend/src/main/java/ma/fsb/sgrh/controller/DocumentController.java` : gère l'upload et la consultation des documents.
- `backend/src/main/java/ma/fsb/sgrh/entity/Document.java` : représente les métadonnées d'un document.
- `backend/src/main/java/ma/fsb/sgrh/repository/DocumentRepository.java` : permet l'accès aux documents stockés.
- `uploads/` : dossier où les fichiers envoyés peuvent être sauvegardés.
- `Tesseract OCR` : composant externe prévu pour l'extraction du texte.

3.7 Interface Analyse IA

3.7.1 Rôle de l'interface

L'interface Analyse IA est destinée aux administrateurs et aux responsables RH. Elle représente la partie intelligente du système et montre comment les données RH peuvent être exploitées au-delà de la simple consultation. L'objectif de ce module est d'utiliser des algorithmes d'apprentissage automatique pour identifier des situations inhabituelles ou produire des regroupements d'employés selon certains comportements.

Dans le cadre du projet, le microservice IA repose sur FastAPI et utilise des bibliothèques Python telles que Scikit-learn, Pandas et NumPy. Le backend Spring Boot communique avec ce microservice à travers des requêtes HTTP. Cette séparation permet de conserver le backend principal en Java tout en profitant de l'écosystème Python, qui est très adapté aux traitements de données et à l'intelligence artificielle.

Les anomalies peuvent concerner des retards inhabituels, des absences répétées, des comportements atypiques ou d'autres éléments détectés par les algorithmes. Par exemple, si un employé présente un nombre de retards très supérieur à la moyenne, le système peut signaler ce comportement comme une anomalie potentielle. L'objectif n'est pas de remplacer la décision humaine, mais de fournir un outil d'aide à l'analyse qui attire l'attention des responsables RH sur des situations particulières.

Le module IA peut également servir à segmenter les profils des employés. Cette segmentation permet de regrouper des individus ayant des comportements similaires, ce qui peut aider à mieux comprendre les tendances internes. Dans un contexte professionnel, ces analyses peuvent être utiles pour anticiper les risques, améliorer le suivi RH et appuyer la prise de décision.

3.7.2 Fichiers en arrière-plan

- `frontend/src/pages/IAPage.jsx` : affiche les actions d'analyse IA, les résultats et les anomalies.
- `frontend/src/services/api.js` : contient `iaService.detecterAnomalies`, `segmenter`, `getRapport` et `getAnomalies`.
- `backend/src/main/java/ma/fsb/sgrh/controller/IAController.java` : communique avec le microservice IA et expose les endpoints IA côté backend.
- `backend/src/main/java/ma/fsb/sgrh/entity/Anomalie.java` : représente une anomalie détectée.
- `backend/src/main/java/ma/fsb/sgrh/repository/AnomalieRepository.java` : permet de récupérer les anomalies.
- `microservice-ia/main.py` : contient l'API FastAPI et les algorithmes IA.

3.8 Interface de gestion des utilisateurs

3.8.1 Rôle de l'interface

L'interface de gestion des utilisateurs est réservée aux administrateurs. Elle permet de consulter les comptes utilisateurs, leurs rôles, leur statut et leur dernière connexion. Ce module est indispensable pour administrer les accès au système, car chaque utilisateur doit être identifié par un compte et rattaché à un rôle précis.

La gestion des utilisateurs permet également de créer, modifier ou supprimer des comptes. Lors de la création d'un compte, l'administrateur définit l'adresse email, le mot de passe, le rôle et le statut du compte. La modification permet de corriger une information ou de changer un rôle si les responsabilités de l'utilisateur évoluent. La suppression permet de retirer un compte qui n'est plus nécessaire, par exemple lorsqu'un employé quitte l'organisation.

Ce module participe directement à la sécurité globale de l'application. En contrôlant les comptes actifs et les rôles associés, l'administrateur limite les accès non autorisés et réduit les risques liés à la consultation ou à la modification de données sensibles.

Les rôles disponibles sont :

- **ADMIN** : accès complet à l'application ;
- **RH** : accès aux modules RH opérationnels ;
- **EMPLOYE** : accès limité aux fonctionnalités personnelles.

3.8.2 Fichiers en arrière-plan

- `frontend/src/pages/UsersPage.jsx` : affiche les comptes, les statistiques et la modale de création/modification.
- `frontend/src/services/api.js` : utilise principalement les services d'authentification pour la création de comptes.
- `backend/src/main/java/ma/fsb/sgrh/controller/AuthController.java` : contient l'inscription d'un utilisateur.
- `backend/src/main/java/ma/fsb/sgrh/entity/Utilisateur.java` : représente un utilisateur.

- `backend/src/main/java/ma/fsb/sgrh/entity/Role.java` : définit les rôles.
- `backend/src/main/java/ma/fsb/sgrh/repository/UtilisateurRepository.java` : permet l'accès aux comptes utilisateurs.

Chapitre 4

Navigation et sécurité des interfaces

4.1 Gestion des routes

La navigation dans l'application repose sur un système de routes défini côté frontend. Les routes de l'application sont déclarées dans le fichier `frontend/src/App.jsx`. Chaque page est associée à une URL précise, ce qui permet à l'utilisateur d'accéder directement aux différents modules après authentification.

La sécurité de navigation est assurée par le composant `ProtectedRoute`. Ce composant vérifie si un utilisateur est connecté avant d'afficher une page protégée. Il vérifie également le rôle de l'utilisateur lorsque la page nécessite une autorisation particulière. Ainsi, si un employé tente d'accéder à la page de gestion des utilisateurs, il est automatiquement redirigé vers le tableau de bord.

Cette organisation améliore la protection de l'application côté interface. Elle ne remplace pas la sécurité backend, mais elle renforce l'expérience utilisateur en évitant d'afficher des pages ou des boutons qui ne correspondent pas aux droits de l'utilisateur connecté.

Chaque page est associée à une URL :

Route	Interface	Rôles autorisés
/login	Connexion	Tous
/dashboard	Tableau de bord	ADMIN, RH, EMPLOYE
/employes	Gestion des employés	ADMIN, RH
/conges	Gestion des congés	ADMIN, RH, EMPLOYE
/presences	Gestion des présences	ADMIN, RH, EMPLOYE
/paie	Gestion de la paie	ADMIN, RH
/documents	Documents et OCR	ADMIN, RH, EMPLOYE
/ia	Analyse IA	ADMIN, RH
/utilisateurs	Gestion des utilisateurs	ADMIN

4.2 Menu latéral

Le menu de navigation est défini dans `frontend/src/components/Sidebar.jsx`. Il constitue l'élément principal permettant à l'utilisateur de se déplacer entre les différentes interfaces. Le menu est affiché dans la mise en page générale de l'application et reste disponible lorsque l'utilisateur navigue d'une page à une autre.

La particularité de ce menu est qu'il filtre automatiquement les entrées selon le rôle de

l'utilisateur connecté. Ainsi, un employé ne voit pas les mêmes modules qu'un administrateur. Cette adaptation dynamique permet de simplifier l'interface et d'éviter d'afficher des fonctionnalités inutiles ou interdites. Elle contribue également à renforcer la cohérence du système de permissions.

Chapitre 5

Guide de lancement de l'application

5.1 Prérequis

Avant de lancer l'application, certains outils doivent être installés sur la machine. Ces outils correspondent aux différentes technologies utilisées dans le projet. Le frontend nécessite Node.js pour installer les dépendances JavaScript et démarrer le serveur de développement Vite. Le backend nécessite Java 21 et Maven pour compiler et exécuter l'application Spring Boot. Le microservice IA nécessite Python 3.11 et les bibliothèques définies dans le fichier des dépendances. Enfin, MySQL est nécessaire pour stocker les données de manière persistante.

Avant de lancer l'application, les outils suivants doivent être disponibles :

- Node.js version 18 ou supérieure ;
- Java JDK 21 ;
- Maven ;
- Python 3.11 ou supérieur ;
- MySQL 8 ;
- Tesseract OCR 5, optionnel mais recommandé pour l'OCR.

5.2 Préparation de la base de données

La base de données constitue un composant fondamental du système, car elle conserve toutes les informations manipulées par l'application. Avant de démarrer le backend, il est donc nécessaire de s'assurer que MySQL fonctionne correctement et que la base `sgrh_intelligent` existe. Le projet fournit deux scripts SQL : le premier permet de créer la structure des tables, tandis que le second permet d'insérer des données initiales de démonstration.

Il faut d'abord démarrer MySQL, puis créer et alimenter la base de données à partir des scripts SQL du projet.

1. Ouvrir un terminal PowerShell ;
2. Vérifier que MySQL est lancé sur le port 3306 ;
3. Exécuter le script `database/schema.sql` pour créer la base `sgrh_intelligent` ;
4. Exécuter le script `database/seed.sql` pour insérer les données de démonstration.

Exemple de commande :

```
mysql -u root -p < database/schema.sql  
mysql -u root -p sgrh_intelligent < database/seed.sql
```

Dans la configuration actuelle, le backend utilise l'utilisateur `root` avec le mot de passe `root`, comme indiqué dans le fichier :

```
backend/src/main/resources/application.yml
```

5.3 Lancement du microservice IA

Le microservice IA doit être lancé séparément du backend principal. Ce choix architectural permet d'isoler les traitements d'intelligence artificielle dans un service spécialisé. Le backend Java peut ainsi appeler ce service uniquement lorsque des analyses sont nécessaires. Cette séparation améliore la modularité du projet et permet de faire évoluer les algorithmes IA indépendamment du reste de l'application.

Depuis le dossier `microservice-ia`, installer les dépendances Python puis lancer FastAPI :

```
py -3.11 -m pip install -r requirements.txt  
py -3.11 -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Le microservice IA doit répondre sur :

```
http://localhost:8000
```

5.4 Lancement du backend Spring Boot

Le backend Spring Boot représente le coeur métier de l'application. Il doit être lancé après la base de données afin de pouvoir établir correctement la connexion MySQL au démarrage. Il expose les endpoints REST utilisés par le frontend, applique les règles de sécurité, gère l'authentification et réalise les opérations de lecture ou d'écriture dans la base de données.

Depuis le dossier `backend`, lancer l'application Spring Boot avec Maven :

```
mvn spring-boot:run
```

Si Maven est installé localement dans le projet, il est possible d'utiliser le chemin complet vers `mvn.cmd`. Le backend doit répondre sur :

```
http://localhost:8080
```

5.5 Lancement du frontend React

Le frontend React correspond à l'interface visible par l'utilisateur. Il doit être lancé après le backend pour que les appels API puissent fonctionner correctement. Pendant le développement, Vite fournit un serveur local rapide qui permet d'afficher l'application dans le navigateur et de recharger automatiquement les modifications.

Depuis le dossier **frontend**, installer les dépendances puis démarrer Vite :

```
npm install  
npm run dev
```

Le frontend doit être accessible depuis le navigateur à l'adresse :

`http://localhost:3000`

5.6 Ordre recommandé de lancement

1. Démarrer MySQL ;
2. Vérifier que la base `sgrh_intelligent` existe ;
3. Démarrer le microservice IA sur le port 8000 ;
4. Démarrer le backend Spring Boot sur le port 8080 ;
5. Démarrer le frontend React sur le port 3000 ;
6. Ouvrir `http://localhost:3000` dans le navigateur.

Chapitre 6

Guide d'utilisation de l'application

6.1 Connexion

Pour utiliser l'application, l'utilisateur doit accéder à la page de connexion, saisir son email et son mot de passe, puis cliquer sur le bouton de connexion. Si les identifiants sont corrects, l'application enregistre les informations de session et redirige l'utilisateur vers le tableau de bord. Dans le cas contraire, un message d'erreur l'informe que la connexion a échoué.

L'expérience utilisateur a été pensée pour être simple : après la connexion, l'utilisateur n'a accès qu'aux modules correspondant à son rôle. Cela évite la confusion et permet à chaque profil de se concentrer sur les tâches qui lui sont réellement destinées.

Comptes de démonstration possibles :

Email	Mot de passe	Rôle
admin@sgrh.ma	password123	ADMIN
rh@sgrh.ma	password123	RH
ahmed.benali@sgrh.ma	password123	EMPLOYE

6.2 Utilisation par un administrateur

L'administrateur dispose de l'accès le plus complet. Il représente le profil ayant la responsabilité de gérer la configuration générale du système et les accès utilisateurs. Il peut consulter tous les modules, intervenir sur les employés, gérer les comptes et accéder aux fonctionnalités sensibles. Ce rôle doit donc être attribué uniquement aux personnes responsables de l'administration de la plateforme.

Dans l'application, l'administrateur peut :

- consulter le tableau de bord ;
- gérer les employés ;
- consulter et traiter les congés ;
- consulter les présences ;
- gérer la paie ;
- gérer les documents ;
- lancer les analyses IA ;
- gérer les comptes utilisateurs.

6.3 Utilisation par un responsable RH

Le responsable RH peut gérer les opérations courantes des ressources humaines. Son rôle est centré sur le suivi du personnel, la gestion des congés, la consultation des présences, l'accès aux documents et l'utilisation des outils d'analyse. Il ne dispose pas nécessairement de toutes les permissions techniques de l'administrateur, mais il possède les fonctionnalités nécessaires au pilotage RH quotidien.

Dans l'application, le responsable RH peut effectuer les opérations suivantes :

- consultation du tableau de bord ;
- gestion des employés ;
- validation ou refus des congés ;
- suivi des présences ;
- consultation de la paie ;
- gestion des documents ;
- consultation des résultats IA.

6.4 Utilisation par un employé

L'employé dispose d'un accès limité afin de protéger les données globales de l'organisation. Son utilisation est principalement orientée vers la consultation de ses informations personnelles, le dépôt de demandes et l'accès aux documents qui le concernent. Ce niveau d'accès respecte le principe du moindre privilège, selon lequel un utilisateur ne doit accéder qu'aux informations nécessaires à ses besoins.

Dans l'application, l'employé peut principalement :

- consulter son tableau de bord ;
- déposer une demande de congé ;
- consulter les présences ;
- déposer ou consulter des documents selon les autorisations.

6.5 Bonnes pratiques d'utilisation

- Toujours se connecter avec un compte correspondant au rôle souhaité ;
- Vérifier les informations avant de valider une modification ;
- Confirmer la suppression uniquement si l'élément n'est plus nécessaire ;
- Démarrer le backend avant d'utiliser les fonctionnalités connectées à la base de données ;
- Démarrer le microservice IA avant de lancer les analyses intelligentes ;
- Vérifier que MySQL fonctionne avant de démarrer le backend.

Chapitre 7

Conclusion

L'application SGRH Intelligent constitue une solution complète pour la gestion moderne des ressources humaines. Elle combine les fonctionnalités classiques d'un système RH avec des modules avancés tels que l'OCR et l'intelligence artificielle. Cette combinaison permet de répondre à la fois aux besoins administratifs habituels et aux besoins plus avancés d'analyse, d'automatisation et d'aide à la décision.

Sur le plan technique, le projet met en oeuvre une architecture web moderne séparant clairement le frontend, le backend, la base de données et le microservice IA. Cette organisation rend l'application plus maintenable et plus évolutive. Elle permet également de remplacer ou d'améliorer un composant sans devoir reconstruire l'ensemble du système. Par exemple, les algorithmes IA peuvent être enrichis dans le microservice Python sans modifier les interfaces React.

Sur le plan fonctionnel, l'application propose une gestion centralisée des employés, des congés, des présences, de la paie, des documents et des utilisateurs. Elle améliore la traçabilité des opérations RH et réduit les risques d'erreurs liés aux traitements manuels. Elle constitue donc une base solide qui pourrait être enrichie dans le futur par d'autres fonctionnalités comme les notifications, les rapports PDF automatiques, l'historique détaillé des actions ou l'intégration avec d'autres systèmes d'information.

Ce guide permet de comprendre les interfaces de l'application, les fichiers techniques associés à chaque module, ainsi que la procédure de lancement et d'utilisation. Il peut être intégré comme une partie descriptive du rapport de Projet de Fin d'Études, car il présente à la fois la vision fonctionnelle de l'application et son organisation technique interne.